ANALOG DEVICES

AHEAD OF WHAT'S POSSIBLE™

# SPI Slave Controller

## Implementation using Verilog HDL

Design Report

By Ahmed H. Ashour

# Introduction

**Serial Peripheral Interface (SPI)** is a widely used synchronous serial communication protocol, commonly used in embedded systems to connect microcontrollers with peripheral devices such as sensors, memory chips and displays unlike asynchronous protocols SPI operates with a shared clock signal enabling faster and more reliable data transfer.

SPI is valued for its simplicity and high speed performance compared to other serial protocols while I2C supports multi master configurations and address based communication it trades off speed and complexity UART is asynchronous and good for long distance data exchange but lacks SPI s simultaneous full duplex transmission SPI s 4 wire design (MOSI, MISO, SCLK and CS) makes it efficient for short range and high speed data exchange where low latency is essential.

SPI play a significant role in system on chip architectures where integration of processors, memory, and peripherals requires fast and predictable communication interfaces. SPI is used to connect digital sensors, external flash memory where minimal overhead and timing are critical.

# Technical Background

The Serial Peripheral Interface (SPI) enables fast and full-duplex data exchange between a master device and one or more slave devices. Its widely adopted in digital systems where reliability, speed, and low communication are essential.

SPI communication is based on 4 signals :

- **MOSI (Master Out, Slave In)** carries data from the master to the slave.
- **MISO (Master In, Slave Out)** carries data from the slave to the master.
- **SCLK (Serial Clock)** generated by the master to synchronize the data transmission.
- **CS (Chip Select)** activated by the master to select which slave should respond.

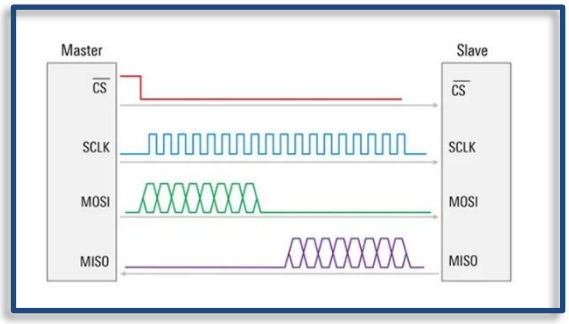


*Figure 1 : SPI master and slave diagram*

the master starts communication by pulling the CS line low, activating the target slave. Then using the clock signal (SCLK) it controls the timing of data exchange sending bits via MOSI and simultaneously receiving bits via MISO. Each transfer is typically 8 bits long. (other sizes are possible depending on the implementation).

SPI operates using different clocking modes, determined by clock polarity (CPOL) and clock phase (CPHA). These modes define whether data is sampled or changed on rising or falling edges of SCLK. **Mode 0** is used In this project, **the clock idles low, data changes on the falling edge, and is sampled on the rising edge**.

## Implementation

This SPI slave module is designed to interface directly with an SPI master and a memory block. It listens to the master signals SCLK, CSB, and SDI while driving SDO back to the master. It starts with a header that contains a read/write flag and an address the based on this header the slave prepares to accept data from the master and write it into memory or it fetches data from memory and shifts it out to the master. Based on SPI mode CPOL=0, CPHA=0.

## State Machine

The control logic has two states HEADER and DATA. In the HEADER state the slave shifts in the header bits from SDI on each rising edge of SCLK the header consists of one bit for the read/write flag and the remaining bits for the memory address. when all header bits are received, the address is latched into the addr register and the state machine goes into the DATA state. reset or deselection (CSB=1) forces the system back into HEADER and resets counters and disables write enable.

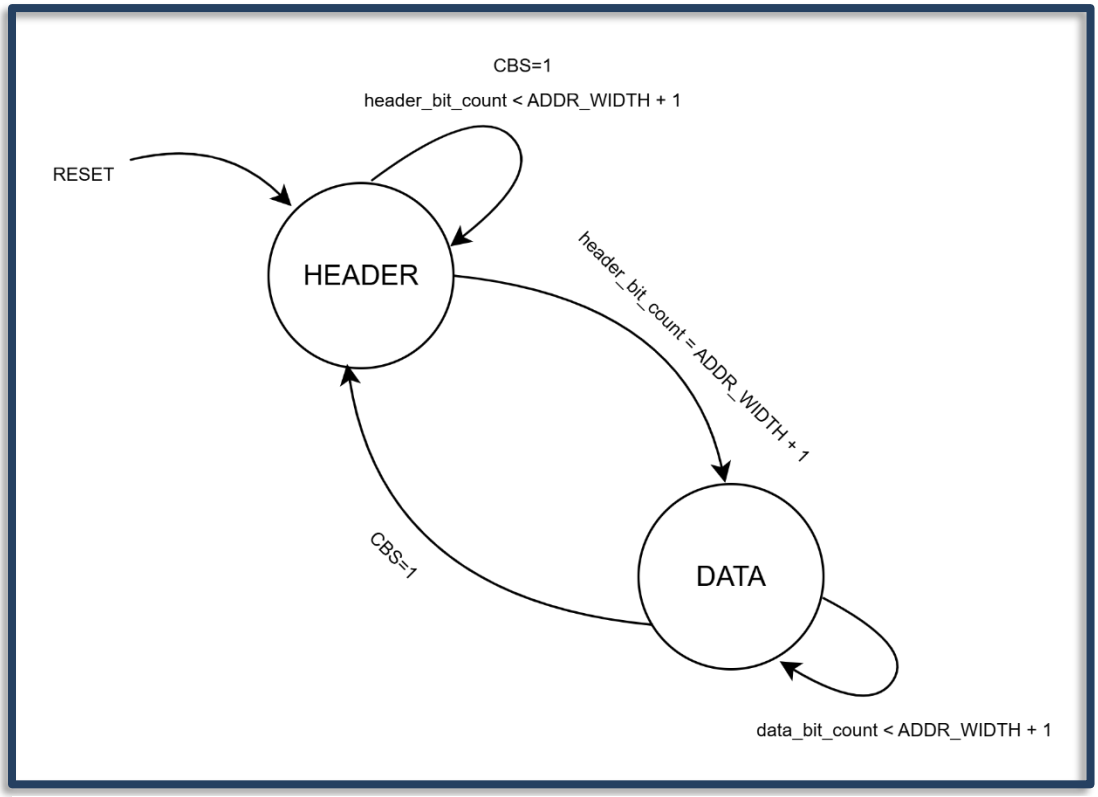


*Figure 2 : SPI slave state diagram*

# HDL Coding

```
module spi_slave #(
parameter DATA_WIDTH = 8,
parameter ADDR_WIDTH = 15
 )(
input rst_n,
//with master
input SCLK,CSB,SDI,
output reg SDO,
//with memory
input [DATA_WIDTH-1:0] rd_data,
output reg wr_en,
output reg [DATA_WIDTH-1:0] wr_data,
output reg [ADDR_WIDTH-1:0] addr
);

//assuming CPOL (clock polarity) = 0 and CPHA (clock phase) = 0,
//so the data is sampled at the rising edge of the clock
//and the data is shifted out at the falling edge of the clock

typedef enum logic {
HEADER= 1'b0,
DATA= 1'b1
} states;

states current_state,next_state;


reg [$clog2(DATA_WIDTH):0] data_bit_counter; // data bit count
reg [$clog2(ADDR_WIDTH)+1:0] header_bit_counter; //wr+address count
reg [DATA_WIDTH-1:0] data_reg;
reg [ADDR_WIDTH:0] header_reg;

//state reg
always @(posedge SCLK or negedge CSB or negedge rst_n or posedge CSB) begin
  if ((rst_n ==0)|(CSB==1'b1)) begin //when the slave is not selected
    current_state= HEADER;
    header_bit_counter= 0;
    data_bit_counter  = 0;
    wr_en = 0;
    SDO=1'bz;
    next_state = HEADER;
  end else begin
    current_state = next_state;
  end
end
```

```
//next state logic (sampling on positive edge)
always @(posedge SCLK) begin
  if (CSB==0)begin //if slave is selected
  case (current_state)
    HEADER:begin //start receiving the header
        SDO=1'bz;
        header_reg={header_reg[ADDR_WIDTH-1:0],SDI};//shift in the header bits from sdi
        header_bit_counter=header_bit_counter+1;//increse the counter
        next_state <= HEADER;//repeat
        if (header_bit_counter == (ADDR_WIDTH+1))begin //when the counter reaches all the bits
        addr=header_reg[ADDR_WIDTH-1:0];//pass adderess to mem
        next_state <= DATA;//go to data state
        end
    end
    DATA:begin
      if (header_reg[ADDR_WIDTH]==1'b0) begin //master write
      wr_en=1'b0;//deafalt value for wr_en
        begin
         data_reg= {data_reg[DATA_WIDTH-1:0],SDI};//shift the data in from sdi
         data_bit_counter <= data_bit_counter+1;//increase the counter
         next_state <= DATA;//repeat
        if (data_bit_counter == (DATA_WIDTH-1)) begin //if a whole byte is shifted in
        wr_en=1'b1;//write enable to mem
        wr_data=data_reg;//write data
        data_bit_counter <= 0;//reset the counter
        //increase the address by 1(for each byte) in case of bursts
        header_reg[ADDR_WIDTH-1:0]<=header_reg[ADDR_WIDTH-1:0]+(DATA_WIDTH/8);
        addr<= header_reg[ADDR_WIDTH-1:0];//pass the new address to mem
        next_state <= DATA;//repeat
        end
        end
      end else begin //master read (the data should change on negative edge)
      wr_en=1'b0;//read signal for mem
        if (data_bit_counter == (DATA_WIDTH)) begin//if all the data is shifted out
        data_bit_counter <= 0;//reset the counter
         //increase the address by 1(for each byte) in case of bursts
        header_reg[ADDR_WIDTH-1:0]<=header_reg[ADDR_WIDTH-1:0]+(DATA_WIDTH/8);
        addr<=header_reg[ADDR_WIDTH-1:0]+(DATA_WIDTH/8);//pass the new address to mem
        next_state <= DATA;//repeat
        end else begin
        next_state <= DATA;
        end
      end
  end
default: next_state = HEADER;
endcase
end
else begin //if not selected
data_bit_counter <= 0;
next_state <= HEADER;
SDO=1'bz;
end
end
//next state logic (driving sdo on negative edge)
always@(negedge SCLK)begin
  if((next_state==DATA)&(header_reg[ADDR_WIDTH]==1'b1)&(CSB==0))begin
  SDO <= rd_data[DATA_WIDTH-data_bit_counter-1];
  data_bit_counter <= data_bit_counter+1;
  end
end
endmodule
```

*Figure 3 : SPI Slave code*

```
module register_map #(
parameter DATA_WIDTH = 8,
parameter ADDR_WIDTH = 15,
parameter DEPTH = (1<<ADDR_WIDTH)
)(
input  clk,wr_en,
input  [ADDR_WIDTH-1:0] addr,
input  [DATA_WIDTH-1:0] wr_data,
output [DATA_WIDTH-1:0] rd_data
);
reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];

always @(negedge clk) begin
    if (wr_en) begin
    mem[addr] <= wr_data;
    end
end
//combinational read
assign rd_data = mem[addr];
endmodule
```

*Figure 4 : register map code*

```
module spi_slave_top #(
parameter DATA_WIDTH = 8,
parameter ADDR_WIDTH = 15
)(
input  rst_n,SCLK,CSB,SDI,
output SDO
);
wire wr_en;
wire [DATA_WIDTH-1:0] wr_data;
wire [ADDR_WIDTH-1:0] addr;
wire [DATA_WIDTH-1:0] rd_data;

spi_slave #(.DATA_WIDTH(DATA_WIDTH),.ADDR_WIDTH(ADDR_WIDTH))
spi_inst (.rst_n(rst_n),.SCLK(SCLK),.CSB(CSB),.SDI(SDI),.SDO(SDO),
.rd_data(rd_data),.wr_en(wr_en),.wr_data(wr_data),.addr(addr));

register_map #(.DATA_WIDTH(DATA_WIDTH),.ADDR_WIDTH(ADDR_WIDTH))
register_map_inst (.clk(SCLK),.wr_en(wr_en),.addr(addr),.wr_data(wr_data),.rd_data(rd_data));

endmodule
```

*Figure 5 : Top Module code*

# Testbench

This testbench drives the spi_slave_top first by reset then write and read transactions to verify its behavior. It begins by holding reset low and keeping the slave deselected with CSB=1, then releases reset and selects the slave after a short delay. A clock generator is used but gated by clk_en so the SPI clock only runs during active transactions The first sequence performs a three byte write to address 0,1 and 2 so the header is sent with the write flag and a zero address, followed by three data bytes shifted in bit by bit on the negative edge of the clock simulating burst writes after the last byte the clock is stopped and after t_lag delay the slave is deselected

```
//select slave
#10
CSB = 0;

//wait t_lead before starting clock
#10 clk_en=1;

//3 byte write operation

SDI=0; //write
@(posedge clk);
clk_en=1;

repeat (ADDR_WIDTH) begin
@(negedge clk);//data changes at the negative edge
SDI = 0; //zero address
end

//test burst by sending multiple bytes
//byte1
repeat (DATA_WIDTH) begin
@(negedge clk);//data changes at the negative edge
SDI = $random; // random data
end
//byte2
repeat (DATA_WIDTH) begin
@(negedge clk);//data changes at the negative edge
SDI = $random; // random data
end
//byte3
repeat (DATA_WIDTH) begin
@(negedge clk);//data changes at the negative edge
SDI = $random; // random data
end

//stop clock after reading the last bit
@(negedge clk);
clk_en = 0;

//wait t_lag then deselect slave
#10 CSB = 1;
```

*Figure 6 : Burst Writing 3 bytes test*

The second sequence performs a three byte read the header is sent with the read flag and the same zero address then the master simply toggles the clock while the slave drives SDO to output three bytes in burst mode. then clock is stopped and the slave is deselected.

```
//select slave
#10
CSB = 0;

//wait t_lead before starting clock
#10 clk_en=1;

//3 byte read operation

SDI=1; //read
@(posedge clk);
clk_en=1;

repeat (ADDR_WIDTH) begin
@(negedge clk);//data changes at the negative edge
SDI = 0; //zero address
end

//byte1
repeat (DATA_WIDTH) begin
@(negedge clk);//data changes at the negative edge
end
//byte2
repeat (DATA_WIDTH) begin
@(negedge clk);//data changes at the negative edge
end
//byte3
repeat (DATA_WIDTH) begin
@(negedge clk);//data changes at the negative edge
end

//stop clock after reading the last bit
@(negedge clk);
clk_en = 0;

//wait t_lag then deselect slave
#10 CSB = 1;
```

*Figure 7 : Burst Reading the Written bytes test*

# Results

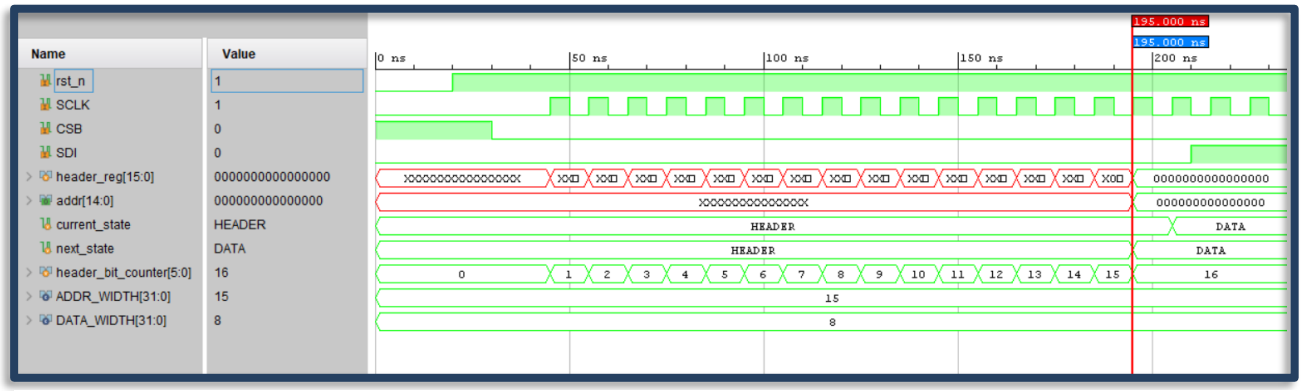


*Figure 8 : Slave reading the header and outputs the correct address*

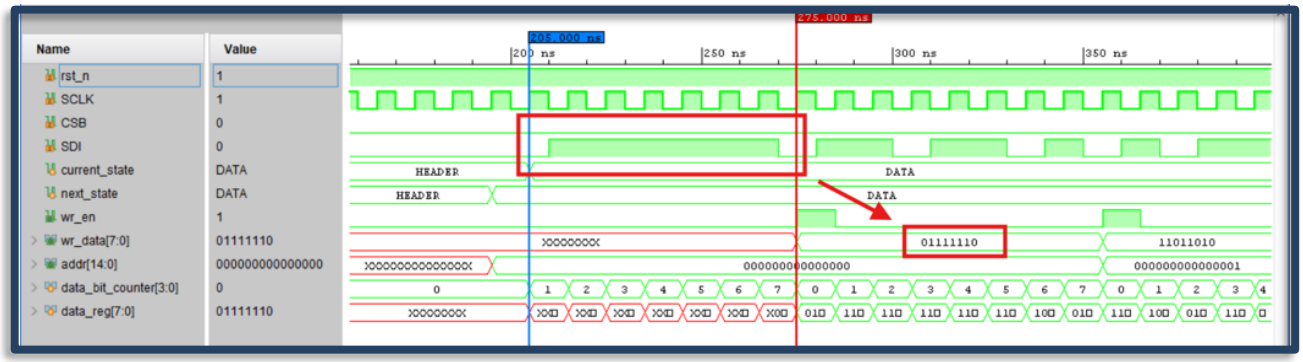


*Figure 9 : slave reads the first byte and writes it to memory at address 0*

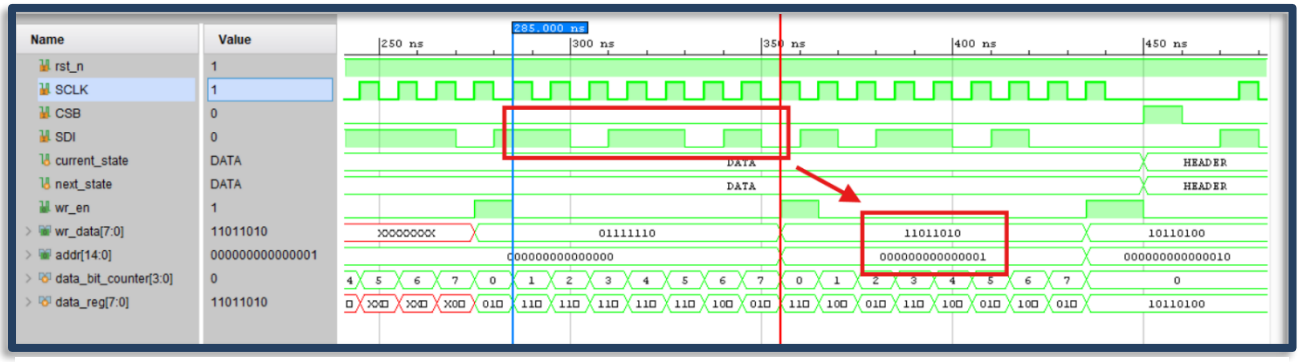


*Figure 10 : the slave reads the next byte and increments the address by one*

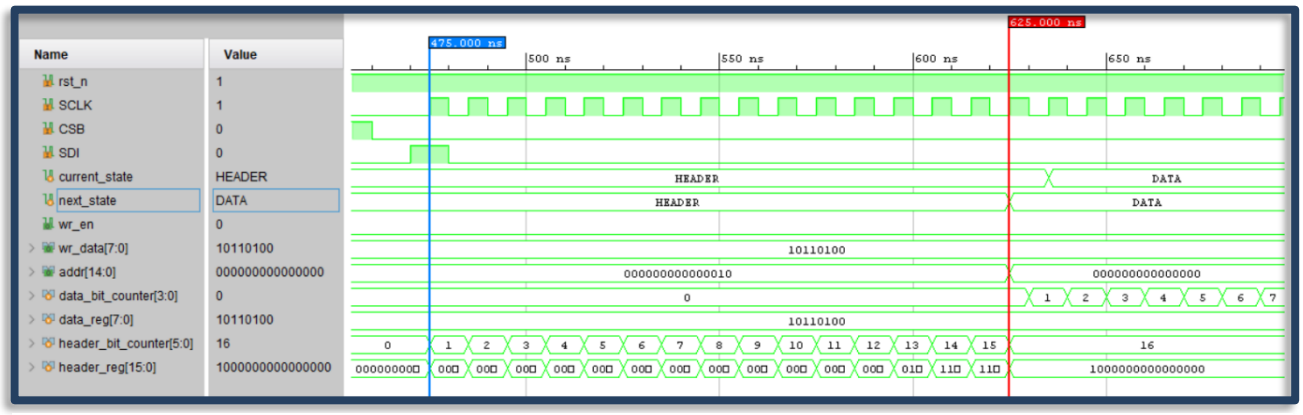


*Figure 11 : Slave reading the header (with read command) and outputs the correct address*

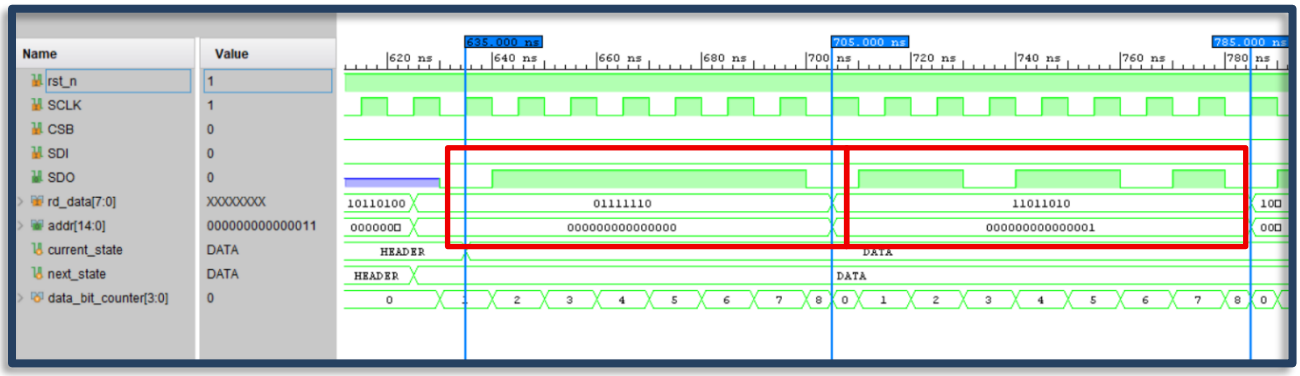


*Figure 12 : Slave drives the SDO with the data from the memory based on the headers address the increments it by one*